Vishal Acharya

Q1. List Aliasing and Mutability

```
a = [10, 20, [30, 40]]
b = a.copy()

b[2].append(50)
b[0] = 100

print(a)
print(b)
```

Concept

`list.copy()` makes a **shallow copy**: a *new outer list* is created, but the elements inside are the *same objects* (references).

- Integers (`10` , `20`) are immutable → replacing `b[0]` just rebinds slot 0 of `b` only.
- The inner list `[30, 40]` is **one shared object** in both `a` and `b` .

Dry run

Step	`a`	`b`
after copy	`[10, 20, L]`	`[10, 20, L]` (same `L = [30,40]`)
`b[2].append(50)`	`L` becomes `[30,40,50]` — visible via both	
`b[0] = 100`	unchanged `10`	`100`

Result: `a = [10, 20, [30, 40, 50]]` , `b = [100, 20, [30, 40, 50]]`

In [1]:
```
a = [10, 20, [30, 40]]
b = a.copy()

print("Is outer list same object? ", a is b)          # False  -> separate outer lists
print("Is inner list same object? ", a[2] is b[2])    # True   -> shared inner list
```

```
b[2].append(50)
b[0] = 100

print(a)
print(b)
```

```
Is outer list same object?  False
Is inner list same object?  True
[10, 20, [30, 40, 50]]
[100, 20, [30, 40, 50]]
```

✅ **Answer: C —** `[10, 20, [30, 40, 50]]` **and** `[100, 20, [30, 40, 50]]`

Q2. Nested List Slicing

```
x = [[10, 20, 30],
     [40, 50, 60],
     [70, 80, 90]]

y = x[1:][::-1]
y[0][1] = 999
print(x)
```

Concept

Slicing a list creates a **new outer list**, but the inner lists are **not copied** — they are still the same objects.

Dry run

1. `x[1:]` → `[[40,50,60], [70,80,90]]` (new list; rows are the *original* row objects)
2. `[::-1]` → reverse → `[[70,80,90], [40,50,60]]`
3. `y[0]` is the row `[70,80,90]` — **the same object as** `x[2]`.
4. `y[0][1] = 999` changes index 1 of that row → `x[2]` becomes `[70, 999, 90]`.

Vishal Acharya

So `x = [[10,20,30],[40,50,60],[70,999,90]]`.

In [2]:
```
x = [[10, 20, 30],
     [40, 50, 60],
     [70, 80, 90]]

y = x[1:][::-1]
print("y before change:", y)
print("y[0] is x[2] ? ", y[0] is x[2])   # True -> same inner list object

y[0][1] = 999
print(x)
```

```
y before change: [[70, 80, 90], [40, 50, 60]]
y[0] is x[2] ?  True
[[10, 20, 30], [40, 50, 60], [70, 999, 90]]
```

✅ **Answer: D —** `[[10,20,30],[40,50,60],[70,999,90]]`

Q3. List Method Interaction

```
L = [5, 2, 8, 2, 9]
x = L.pop(1)
L.insert(2, x)
L.remove(2)
L.reverse()
print(L)
```

Dry run

Statement	What happens	`L`
start		`[5, 2, 8, 2, 9]`
`x = L.pop(1)`	removes **index 1** → `x = 2`	`[5, 8, 2, 9]`
`L.insert(2, x)`	inserts `2` at index 2	`[5, 8, 2, 2, 9]`

Vishal Acharya

Statement	What happens	`L`
`L.remove(2)`	removes the **first value equal to 2** (index 2)	`[5, 8, 2, 9]`
`L.reverse()`	in-place reverse	`[9, 2, 8, 5]`

Actual output: `[9, 2, 8, 5]`

⚠️ Note about the options

`[9, 2, 8, 5]` does **not** appear in options A–F, and there is **no runtime error** (option G is wrong too). The closest option is **D** `[9, 8, 2, 5]`, which differs only by the order of `2` and `8` — so this looks like a typo in the question paper. Mention this to your teacher; if forced to choose, the *intended* answer is most likely **D**, but the verified real output is `[9, 2, 8, 5]`.

Key lesson: `remove(x)` deletes only the **first** occurrence; `pop(i)` removes by **index**.

In [3]:
```
L = [5, 2, 8, 2, 9]
x = L.pop(1);      print("after pop(1)      :", L, " x =", x)
L.insert(2, x);    print("after insert(2,x) :", L)
L.remove(2);       print("after remove(2)   :", L)
L.reverse();       print("after reverse()   :", L)
```

```
after pop(1)      : [5, 8, 2, 9]  x = 2
after insert(2,x) : [5, 8, 2, 2, 9]
after remove(2)   : [5, 8, 2, 9]
after reverse()   : [9, 2, 8, 5]
```

✅ Verified output: `[9, 2, 8, 5]` — not listed (closest: D; likely paper typo)

Q4. List Comprehension

```
result = [x * y
          for x in range(1, 5)
          for y in range(1, 4)
          if (x + y) % 2 == 0]
print(result)
```

Vishal Acharya

Vishal Acharya

Concept

Nested loops in a comprehension read left→right like normal nested `for` loops: **outer** `x` **∈ {1,2,3,4}**, **inner** `y` **∈ {1,2,3}**. Keep the product only when `x + y` is even.

Dry run

x	y	x+y	even?	x·y kept
1	1	2	✔	**1**
1	2	3	✗	
1	3	4	✔	**3**
2	1	3	✗	
2	2	4	✔	**4**
2	3	5	✗	
3	1	4	✔	**3**
3	2	5	✗	
3	3	6	✔	**9**
4	1	5	✗	
4	2	6	✔	**8**
4	3	7	✗	

Result = `[1, 3, 4, 3, 9, 8]` — matches none of A–F.

In [4]:
```
result = [x * y
          for x in range(1, 5)
          for y in range(1, 4)
```

```
            if (x + y) % 2 == 0]
print(result)
```

```
[1, 3, 4, 3, 9, 8]
```

✅ Answer: **G** — None of the above (actual output `[1, 3, 4, 3, 9, 8]`)

Q5. Dictionary Reference vs Copy

```
d1 = {"A": {"x": 10}, "B": 20}
d2 = d1.copy()
d2["A"]["x"] = 100
d2["B"] = 200
print(d1)
```

Concept

`dict.copy()` is also a **shallow copy**.

- `d2["A"]` and `d1["A"]` point to the **same inner dict** → modifying `d2["A"]["x"]` is visible in `d1`.
- `d2["B"] = 200` **rebinds** key `"B"` in `d2` only (ints are immutable) → `d1["B"]` stays `20`.

So `d1 = {'A': {'x': 100}, 'B': 20}`.

In [5]:
```
d1 = {"A": {"x": 10}, "B": 20}
d2 = d1.copy()
print(d1["A"] is d2["A"])    # True -> shared inner dict
d2["A"]["x"] = 100
d2["B"] = 200
print(d1)
print(d2)
```

```
True
{'A': {'x': 100}, 'B': 20}
{'A': {'x': 100}, 'B': 200}
```

Vishal Acharya

✅ Answer: **B** — `{'A': {'x': 100}, 'B': 20}`

Q6. Dictionary Comprehension

```
data = {"A": 10, "B": 15, "C": 20, "D": 25}
result = {k: v // 5 for k, v in data.items() if v % 10 == 0}
print(result)
```

Dry run

Condition `v % 10 == 0` → only 10 and 20 pass (15 and 25 leave remainder 5).

- `A: 10 // 5 = 2`
- `C: 20 // 5 = 4`

`//` is floor (integer) division.

In [6]:
```
data = {"A": 10, "B": 15, "C": 20, "D": 25}
result = {k: v // 5 for k, v in data.items() if v % 10 == 0}
print(result)
```

```
{'A': 2, 'C': 4}
```

✅ Answer: **A** — `{'A': 2, 'C': 4}`

Q7. Set Operations

```
A = {1, 2, 3, 4, 5}
B = {3, 4, 5, 6, 7}
result = (A ^ B) - (A & B)
print(result)
```

Operators

Vishal Acharya

Operator	Meaning	Value here
`A ^ B`	symmetric difference (in exactly one set)	`{1, 2, 6, 7}`
`A & B`	intersection (common)	`{3, 4, 5}`
`-`	difference	`{1,2,6,7} - {3,4,5}`

The symmetric difference already contains **no** common elements, so subtracting the intersection removes nothing → `{1, 2, 6, 7}`.

In [7]:
```
A = {1, 2, 3, 4, 5}
B = {3, 4, 5, 6, 7}
print("A ^ B :", A ^ B)
print("A & B :", A & B)
print("result:", (A ^ B) - (A & B))
```

```
A ^ B : {1, 2, 6, 7}
A & B : {3, 4, 5}
result: {1, 2, 6, 7}
```

✅ **Answer: A** — `{1, 2, 6, 7}`

Vishal Acharya

Vishal Acharya

Q8. Lambda + Filter + Map (+ reduce)

```
from functools import reduce
data = [3, 6, 9, 12, 15]
x = list(map(lambda n: n // 3, filter(lambda n: n % 2 == 0, data)))
result = reduce(lambda a, b: a + b, x)
print(result)
```

Pipeline (inside → outside)

1. **filter** keeps even numbers: `[6, 12]`
2. **map** `n // 3` → `[2, 4]`
3. **reduce** adds cumulatively: `2 + 4 = 6`

In [8]:
```
from functools import reduce
data = [3, 6, 9, 12, 15]

f = list(filter(lambda n: n % 2 == 0, data));  print("filter:", f)
m = list(map(lambda n: n // 3, f));            print("map   :", m)
print("reduce:", reduce(lambda a, b: a + b, m))
```

```
filter: [6, 12]
map   : [2, 4]
reduce: 6
```

✅ Answer: **A** — `6`

Q9. Advanced Lambda

```
data = [("Python",85,90), ("Java",90,80), ("C++",75,95), ("Go",88,92)]
result = sorted(data, key=lambda x: x[1] + x[2], reverse=True)
print(result[0][0])
```

Dry run

Key = sum of the two scores: | Language | Sum | |---|---| | Python | 175 | | Java | 170 | | C++ | 170 | | Go | **180** |

Descending order → Go first. `result[0][0]` = name of first tuple = `"Go"`.

In [9]:
```
data = [("Python",85,90), ("Java",90,80), ("C++",75,95), ("Go",88,92)]
result = sorted(data, key=lambda x: x[1] + x[2], reverse=True)
print(result)
print(result[0][0])
```

```
[('Go', 88, 92), ('Python', 85, 90), ('Java', 90, 80), ('C++', 75, 95)]
Go
```

✅ **Answer: D — Go**

Q10. Mixed List–Dictionary–Set Problem

```
data = [{"name":"A","marks":[80,90,70]},
        {"name":"B","marks":[90,90,80]},
        {"name":"C","marks":[70,80,90]}]

result = {d["name"]: sum(d["marks"]) // len(d["marks"])
          for d in data if max(d["marks"]) >= 90}
print(result)
```

Dry run

Name	max	passes `max >= 90`?	sum // len
A	90	✔	240 // 3 = **80**
B	90	✔	260 // 3 = **86** (floor of 86.67)
C	90	✔	240 // 3 = **80**

All three pass the filter (each has a 90).

Vishal Acharya

Vishal Acharya

In [10]:
```
data = [{"name":"A","marks":[80,90,70]},
        {"name":"B","marks":[90,90,80]},
        {"name":"C","marks":[70,80,90]}]
result = {d["name"]: sum(d["marks"]) // len(d["marks"])
          for d in data if max(d["marks"]) >= 90}
print(result)
```

```
{'A': 80, 'B': 86, 'C': 80}
```

✅ **Answer: A —** `{'A': 80, 'B': 86, 'C': 80}`

Q11. Student Performance Analytics

students = [("Amit", "Python", [78, 85, 92]), ("Rahul", "Python", [88, 76, 95]), ("Neha", "Java", [91, 89, 94]), ("Priya", "Python", [65, 72, 70]), ("Karan", "Java", [82, 85, 88])]

1. Calculate the average marks of every student.
2. Create a dictionary in the form student_name : average.
3. Identify students whose average is greater than or equal to 80.
4. Store their names in a set.
5. Find the student having the highest average using lambda.
6. Display students in descending order of average.
7. Display the unique subjects using a set.
8. Create the final result using an appropriate dictionary comprehension.

In [21]:
```
students = [
    ("Amit",  "Python", [78, 85, 92]),
    ("Rahul", "Python", [88, 76, 95]),
    ("Neha",  "Java",   [91, 89, 94]),
    ("Priya", "Python", [65, 72, 70]),
    ("Karan", "Java",   [82, 85, 88])
]

# 1 & 2) Average of every student  ->  dictionary {name: average}
averages = {name: sum(marks) / len(marks) for name, subject, marks in students}
```

Vishal Acharya

```python
print("1-2) Averages          :", {k: round(v, 2) for k, v in averages.items()})

# 3 & 4) Students with average >= 80  ->  stored in a SET
top_students = {name for name, avg in averages.items() if avg >= 80}
print("3-4) Avg >= 80 (set)   :", top_students)

# 5) Student with the highest average using lambda
best = max(averages.items(), key=lambda item: item[1])   # item = (name, avg)
print("5)   Highest average   :", best[0], "->", round(best[1], 2))

# 6) Descending order of average
ranking = sorted(averages.items(), key=lambda item: item[1], reverse=True)
print("6)   Descending order  :")
for rank, (name, avg) in enumerate(ranking, start=1):
    print(f"      {rank}. {name:<6} {avg:6.2f}")

# 7) Unique subjects using a set
subjects = {subject for _, subject, _ in students}
print("7)   Unique subjects   :", subjects)

# 8) Final result using an appropriate dictionary comprehension
#    (name -> rounded average, only for students who scored >= 80)
final_result = {name: round(avg, 2) for name, avg in averages.items() if avg >= 80}
print("8)   Final dictionary  :", final_result)
```

```
1-2) Averages          : {'Amit': 85.0, 'Rahul': 86.33, 'Neha': 91.33, 'Priya': 69.0, 'Karan': 85.0}
3-4) Avg >= 80 (set)   : {'Rahul', 'Karan', 'Neha', 'Amit'}
5)   Highest average   : Neha -> 91.33
6)   Descending order  :
      1. Neha    91.33
      2. Rahul   86.33
      3. Amit    85.00
      4. Karan   85.00
      5. Priya   69.00
7)   Unique subjects   : {'Python', 'Java'}
8)   Final dictionary  : {'Amit': 85.0, 'Rahul': 86.33, 'Neha': 91.33, 'Karan': 85.0}
```

Explanation of key points

- `sum(marks)/len(marks)` → true division gives a float average (e.g. Rahul = 259/3 = 86.33).
- A **set** removes duplicates & has no order — ideal for "unique" and membership results.

- `max(..., key=lambda item: item[1])` compares dict items by their *value* (average).
- `sorted(..., reverse=True)` returns a **new** list in descending order; Amit and Karan tie at 85.0 and keep original order (Python's sort is *stable*).

Q12. Inventory Analysis System

Formula: `total_value = price × quantity` .

Q12. Inventory Analysis System

inventory = {

```
    "Laptop": {"price": 55000, "quantity": 8, "category": "Electronics"},

    "Mobile": {"price": 25000, "quantity": 15, "category": "Electronics"},

    "Chair": {"price": 3500, "quantity": 20, "category": "Furniture"},

    "Table": {"price": 7000, "quantity": 10, "category": "Furniture"}
```

}

9. Calculate the total inventory value of every product.
10. Create a dictionary containing product : total_value.
11. Find products whose total inventory value exceeds ₹2,00,000.
12. Find all unique categories using a set.
13. Find the product having maximum inventory value using lambda.
14. Sort products according to inventory value in descending order.
15. Use dictionary comprehension wherever appropriate.

In [1]:
```
inventory = {
    "Laptop": {"price": 55000, "quantity": 8,  "category": "Electronics"},
    "Mobile": {"price": 25000, "quantity": 15, "category": "Electronics"},
    "Chair":  {"price": 3500,  "quantity": 20, "category": "Furniture"},
```

Vishal Acharya

Vishal Acharya

```
    "Table":  {"price": 7000,  "quantity": 10, "category": "Furniture"}
}

# 1 & 2) Total inventory value of each product -> {product: total_value}
total_value = {item: d["price"] * d["quantity"] for item, d in inventory.items()}
print("1-2) Total value per product :", total_value)

# 3) Products whose total value exceeds Rs 2,00,000
high_value = {item: v for item, v in total_value.items() if v > 200000}
print("3)   Value > 2,00,000        :", high_value)

# 4) Unique categories using a set
categories = {d["category"] for d in inventory.values()}
print("4)   Unique categories       :", categories)

# 5) Product with maximum inventory value using lambda
max_item = max(total_value.items(), key=lambda kv: kv[1])
print("5)   Max inventory value     :", max_item[0], "-> Rs", f"{max_item[1]:,}")

# 6) Sort products by inventory value, descending
sorted_items = sorted(total_value.items(), key=lambda kv: kv[1], reverse=True)
print("6)   Sorted (descending)     :")
for name, val in sorted_items:
    print(f"      {name:<8} Rs {val:>9,}")

# 7) Extra: dict comprehension -> category-wise total value
category_totals = {c: sum(v for p, v in total_value.items() if inventory[p]["category"] == c)
                   for c in categories}
print("7)   Category-wise totals    :", category_totals)
```

```
1-2) Total value per product : {'Laptop': 440000, 'Mobile': 375000, 'Chair': 70000, 'Table': 70000}
3)   Value > 2,00,000        : {'Laptop': 440000, 'Mobile': 375000}
4)   Unique categories       : {'Furniture', 'Electronics'}
5)   Max inventory value     : Laptop -> Rs 440,000
6)   Sorted (descending)     :
      Laptop   Rs   440,000
      Mobile   Rs   375,000
      Chair    Rs    70,000
      Table    Rs    70,000
7)   Category-wise totals    : {'Furniture': 140000, 'Electronics': 815000}
```

In [5]:
```
inventory = {
    "Laptop": {"price": 55000, "quantity": 8,  "category": "Electronics"},
    "Mobile": {"price": 25000, "quantity": 15, "category": "Electronics"},
    "Chair":  {"price": 3500,  "quantity": 20, "category": "Furniture"},
    "Table":  {"price": 7000, "quantity": 10, "category": "Furniture"}
}

inventory = {
    product: {
        "price": details["price"],
        "quantity": details["quantity"],
        "category": details["category"],
        "inventory": details["price"] * details["quantity"]
    }
    for product, details in inventory.items()
}

print(inventory)
```

```
{'Laptop': {'price': 55000, 'quantity': 8, 'category': 'Electronics', 'inventory': 440000}, 'Mobile': {'price': 25000, 'quantit
y': 15, 'category': 'Electronics', 'inventory': 375000}, 'Chair': {'price': 3500, 'quantity': 20, 'category': 'Furniture', 'inv
entory': 70000}, 'Table': {'price': 7000, 'quantity': 10, 'category': 'Furniture', 'inventory': 70000}}
```

Explanation

- Laptop = 55000 × 8 = 4,40,000; Mobile = 25000 × 15 = 3,75,000; Chair = 3500 × 20 = 70,000; Table = 7000 × 10 = 70,000.
- Only Laptop and Mobile exceed ₹2,00,000. Laptop is the maximum.
- Dictionary comprehensions are used in steps 2, 3 and 7; set comprehension in step 4.

Vishal Acharya

Vishal Acharya

T2_CLASS _PROGRAM_VHA_0

November 4, 2025

1 Write a Python program to find the minimum window in a given string which will contain all the characters of another given string. (You can use your own method to find the solution as well)

Input : str1 = " PRWSOERIUSFK "

str2 = " OSU "

Output: Minimum window is "OERIUS

[3]:
```
def minWindow(s,t) :

        counter_t = {}
        counter_s = {}

        for c in t:
            counter_t[c] = counter_t.get(c, 0) + 1


        i = 0
        j = 0

        left = -1
        right = -1

        valid = 0

        for i in range(len(s)):

            while j < len(s) and valid < len(counter_t):
                counter_s[s[j]] = counter_s.get(s[j], 0) + 1

                if s[j] in counter_t and counter_s[s[j]] == counter_t[s[j]]:
                    valid += 1


                j += 1
```

Vishal Acharya

```
            if valid == len(counter_t):
                if left == -1 or j - i < right - left:
                    left = i
                    right = j


            counter_s[s[i]] -= 1
            if s[i] in counter_t and counter_s[s[i]] == counter_t[s[i]] - 1:
                valid -= 1

        if left == -1:
            return "not found"


        return s[left : right]
s=""this is a test string""
t="tist"
print(minWindow(s,t))
s = "PRWSvOERIUSFK"
t = "OSU"
print(minWindow(s,t))
s="abghac bgfabac hgccbbaa"
t="aabc"
print(minWindow(s,t))
```

```
t stri
OERIUS
abac
```

2 Write a program that converts Roman numerals into ordinary numbers. Here are the conversions: M=1000, D=500, C=100, L=50, X=10, V=5 I=1. Don't forget about things like IV being 4 and XL being 40.

3 Write a program that converts ordinary numbers into Roman numerals

[1]:
```
def romanToInt(s):
        """
        :type s: str
        :rtype: int
        """
```

```
        roman = {'I':1,'V':5,'X':10,'L':50,'C':100,'D':500,'M':1000,'IV':4,'IX':
↪9,'XL':40,'XC':90,'CD':400,'CM':900}
        i = 0
        num = 0
        while i < len(s):
            if i+1<len(s) and s[i:i+2] in roman:
                num+=roman[s[i:i+2]]
                i+=2
            else:
                num+=roman[s[i]]
                i+=1
        return num
s=input("Enter number in Roman: ")
print(romanToInt(s))
```

```
Enter number in Roman: LVIII
58
```

[14]:
```
def int_to_Roman(num):
        val = [
            1000, 900, 500, 400,
            100, 90, 50, 40,
            10, 9, 5, 4,
            1
            ]
        syb = [
            "M", "CM", "D", "CD",
            "C", "XC", "L", "XL",
            "X", "IX", "V", "IV",
            "I"
            ]
        roman_num = ''
        i = 0
        while  num > 0:
            for j in range(num // val[i]):
                roman_num += syb[i]
                num -= val[i]
            i += 1
        return roman_num
num=int(input("Input an integer: "))
print(int_to_Roman(num))
```

```
Input an integer: 58
LVIII
```

Vishal Acharya

Vishal Acharya

4 Given an integer value, return a string with the equivalent English text of each digit. For example, an input of 89 results in "eighty-nine" being returned. Restrict values to be between 0 and 10^15.

[16]:
```
def int_to_en(num):
    d = { 0 : 'zero', 1 : 'one', 2 : 'two', 3 : 'three', 4 : 'four', 5 : 'five',
          6 : 'six', 7 : 'seven', 8 : 'eight', 9 : 'nine', 10 : 'ten',
          11 : 'eleven', 12 : 'twelve', 13 : 'thirteen', 14 : 'fourteen',
          15 : 'fifteen', 16 : 'sixteen', 17 : 'seventeen', 18 : 'eighteen',
          19 : 'nineteen', 20 : 'twenty',
          30 : 'thirty', 40 : 'forty', 50 : 'fifty', 60 : 'sixty',
          70 : 'seventy', 80 : 'eighty', 90 : 'ninety' }
    k = 1000
    m = k * 1000
    b = m * 1000
    t = b * 1000

    if (num < 20):
        return d[num]

    if (num < 100):
        if num % 10 == 0:
            return d[num]
        else:
            return d[num // 10 * 10] + '-' + d[num % 10]

    if (num < k):
        if num % 100 == 0:
            return d[num // 100] + ' hundred'
        else:
            return d[num // 100] + ' hundred and ' + int_to_en(num % 100)

    if (num < m):
        if num % k == 0:
            return int_to_en(num // k) + ' thousand'
        else:
            return int_to_en(num // k) + ' thousand, ' + int_to_en(num % k)

    if (num < b):
        if (num % m) == 0:
            return int_to_en(num // m) + ' million'
        else:
            return int_to_en(num // m) + ' million, ' + int_to_en(num % m)

    if (num < t):
```

```
        if (num % b) == 0:
            return int_to_en(num // b) + ' billion'
        else:
            return int_to_en(num // b) + ' billion, ' + int_to_en(num % b)

    if (num >= t):
        if (num % t == 0):
            return int_to_en(num // t) + ' trillion'
        else:
            return int_to_en(num // t) + ' trillion, ' + int_to_en(num % t)
num=int(input("Enter any positive integer: "))
print(int_to_en(num))
```

```
Enter any positive integer: 3669478521369427
three thousand, six hundred and sixty-nine trillion, four hundred and seventy-
eight billion, five hundred and twenty-one million, three hundred and sixty-nine
thousand, four hundred and twenty-seven
```

5 Valid Parentheses

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: s = "()"

Output: true

Example 2:

Input: s = "()"

Output: true

Example 3:

Input: s = "(]"

Output: false

[1]:
```
s="))()"
max_length = 0
stack = [-1]  # Initialize with a start index

for i in range(len(s)):
```

Vishal Acharya

```
    if s[i] == '(':
        stack.append(i)
    else:
        stack.pop()

    if not stack:
        stack.append(i)  # If popped -1, add a new start index
    else:
    # Update the length of the valid substring
        max_length = max(max_length, i - stack[-1])
print(max_length)
```

2

6 CANDY

There are n children standing in a line. Each child is assigned a rating value given in the integer array ratings.

You are giving candies to these children subjected to the following requirements:

Each child must have at least one candy.

Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

Example 1:

Input: ratings = [1,0,2]

Output: 5

Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.

Example 2:

Input: ratings = [1,2,2]

Output: 4

Explanation: You can allocate to the first, second and third child with 1, 2, 1 candies respectively. The third child gets 1 candy because it satisfies the above two conditions.

[5]:
```
ratings=eval(input("ENTER LIST "))
n=len(ratings)
dp=[1]*n
for i in range(1,n):
    if ratings[i]>ratings[i-1]:
                dp[i]=dp[i-1]+1

for i in range(n-2,-1,-1):
    if ratings[i]>ratings[i+1]:
```

Vishal Acharya

```
        dp[i]=max(dp[i],dp[i+1]+1)
print("candy distribution",dp)
print("total candy", sum(dp))
```

```
ENTER LIST [5,2,4,7,1,3,5,8,3,6]
candy distribution [2, 1, 2, 3, 1, 2, 3, 4, 1, 2]
total candy 21
```

7 What Is The Container With Most Water Problem?

Container With the Most Water is a coding problem that involves finding the largest possible area that can be formed by two vertical lines on a graph, bound by the height of the shorter line. This problem can be solved using a two-pointer approach, which involves traversing the array from both sides and keeping track of the maximum area found so far.

You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]).

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: vertical lines are represented by array [1,8,6,2,5,4,8,3,7].

[13]:
```
height=eval(input("enter of list "))
left = 0
right = len(height) - 1
Area = 0
while left < right:
    currentArea = min(height[left], height[right]) * (right - left)
    Area = max(Area, currentArea)
    if height[left] < height[right]:
        left += 1
    else:
        right -= 1

print(Area)
```

```
enter of list [1,8,6,2,5,4,8,3,7]
49
```

Vishal Acharya

8 Trapping Rain Water

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

[15]:
```
height=[0,1,0,2,1,0,1,3,2,1,2,1]
n = len(height)

if n < 1:
    ans=0

ans = 0
left_max = [0] * n
right_max = [0] * n

left_max[0] = height[0]
for i in range(1, n):
    left_max[i] = max(left_max[i-1], height[i])

right_max[-1] = height[-1]
for i in range(n-2, -1, -1):
    right_max[i] = max(right_max[i+1], height[i])

for i in range(n):
    ans += min(left_max[i], right_max[i]) - height[i]
print(ans)
```

```
6
```

Vishal Acharya

Vishal Acharya

T2_CLASS _PROGRAM_VHA

November 4, 2025

1 Get a string from a given string where all occurrences of its first char have been changed to '$', except the first char itself

Sample String : 'restart' Expected Result : 'resta$t'

[1]:
```
def change_char(str1):
  char = str1[0]
  str1 = str1.replace(char, '$')
  str1 = char + str1[1:]

  return str1

print(change_char('restart'))
```

```
resta$t
```

2 Count the occurrences of each word in a given sentence

[2]:
```
def word_count(str):
    counts = dict()
    words = str.split()

    for word in words:
        if word in counts:
            counts[word] += 1
        else:
            counts[word] = 1

    return counts

print( word_count('the quick brown fox jumps over the lazy dog.'))
```

```
{'the': 2, 'quick': 1, 'brown': 1, 'fox': 1, 'jumps': 1, 'over': 1, 'lazy': 1,
'dog.': 1}
```

Vishal Acharya

3 Write a Python program to create a Caesar encryption.

Note: In cryptography, a Caesar cipher, also known as Caesar's cipher, the shift cipher, Caesar's code or Caesar shift, is one of the simplest and most widely known encryption techniques. It is a type of substitution cipher in which each letter in the plaintext is replaced by a letter some fixed number of positions down the alphabet. For example, with a left shift of 3, D would be replaced by A, E would become B, and so on. The method is named after Julius Caesar, who used it in his private correspondence.

key would be in negative if you want to left shift. key would be positive if you want to right shift.

4 1st Method

[59]:
```
s=input()
key=int(input())
s1=""
for i in s:
    if i>='A' and i<='Z':
        if ord(i)+key>ord('Z'):
            i=chr(ord(i)+key-26)
        else:
            i=chr(ord(i)+key)
        s1+= i
    elif i>='a' and i<='z':
        if ord(i)+key>ord('z'):
            i=chr(ord(i)+key-26)
        else:
            i=chr(ord(i)+key)
        s1+= i
    else:
        s1+= i
print(s1)
```

```
thisispython
2
vjkukuravjqp
```

5 2nd Method

[58]:
```
def caesar_encrypt(realText, step):
    outText = []
    cryptText = []

    uppercase = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L',␣
 ↪'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z']
    lowercase = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l',␣
 ↪'m', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
```

```
    for eachLetter in realText:
        if eachLetter in uppercase:
            index = uppercase.index(eachLetter)
            crypting = (index + step) % 26
            cryptText.append(crypting)
            newLetter = uppercase[crypting]
            outText.append(newLetter)
        elif eachLetter in lowercase:
            index = lowercase.index(eachLetter)
            crypting = (index + step) % 26
            cryptText.append(crypting)
            newLetter = lowercase[crypting]
            outText.append(newLetter)
    return ''.join(outText)
code = caesar_encrypt('thisispython', 2)
print()
print(code)
print()
```

```
vjkukuravjqp
```

6 3rd Method

[61]:
```
def encrypt(key, message):
    message = message.upper()
    alpha = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    result = ""

    for letter in message:
        if letter in alpha: #if the letter is actually a letter
            #find the corresponding ciphertext letter in the alphabet
            letter_index = (alpha.find(letter) + key) % len(alpha)

            result = result + alpha[letter_index]
        else:
            result = result + letter

    return result

def decrypt(key, message):
    message = message.upper()
    alpha = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    result = ""
```

Vishal Acharya

```
    for letter in message:
        if letter in alpha: #if the letter is actually a letter
            #find the corresponding ciphertext letter in the alphabet
            letter_index = (alpha.find(letter) - key) % len(alpha)

            result = result + alpha[letter_index]
        else:
            result = result + letter

    return result
message="ywa"
print(encrypt(4, message))
print(decrypt(4, 'cae'))
```

```
CAE
YWA
```

7 Demo of Join Function

[4]:
```
#demo of join function
#if joins all arguments given inside of list of strings to a blank string.
a="hello"
b="LJU"
print(''.join([a,b]))
```

```
helloLJU
```

[5]:
```
#demo of join function
#if joins all arguments given inside of list of strings with space in between.
a="hello"
b="LJU"
c='welcome'
print(' '.join([a,b,c]))
```

```
hello LJU welcome
```

8 Write a Python program to reverse words in a string

[9]:
```
def reverse_string_words(text):
    for line in text.split('\n'):
        return(' '.join(line.split()[::-1]))
print(reverse_string_words("The quick brown fox jumps over the lazy dog."))
print(reverse_string_words("Python Exercises."))
```

```
dog. lazy the over jumps fox brown quick The
Exercises. Python
```

Vishal Acharya

9 Write a Python program to find the second most repeated word in a given string.

[15]:
```
def word_count(str):
    counts = dict()
    words = str.split()

    for word in words:
        if word in counts:
            counts[word] += 1
        else:
            counts[word] = 1

    counts_x = sorted(counts.items(), key=lambda kv: kv[1])
    print(counts_x)
    return counts_x[-2]

print(word_count("Both of these issues are fixed by postponing the evaluation␣
 ↪of annotations. Instead of compiling code which executes expressions in␣
 ↪annotations at their definition time, the compiler stores the annotation in␣
 ↪a string form equivalent to the AST of the expression in question. If␣
 ↪needed, annotations can be resolved at runtime using typing.get_type_hints().
 ↪ In the common case where this is not required, the annotations are cheaper␣
 ↪to store (since short strings are interned by the interpreter) and make␣
 ↪startup time faster."))
```

```
[('Both', 1), ('these', 1), ('issues', 1), ('fixed', 1), ('postponing', 1),
('evaluation', 1), ('annotations.', 1), ('Instead', 1), ('compiling', 1),
('code', 1), ('which', 1), ('executes', 1), ('expressions', 1), ('their', 1),
('definition', 1), ('time,', 1), ('compiler', 1), ('stores', 1), ('annotation',
1), ('a', 1), ('string', 1), ('form', 1), ('equivalent', 1), ('AST', 1),
('expression', 1), ('question.', 1), ('If', 1), ('needed,', 1), ('can', 1),
('be', 1), ('resolved', 1), ('runtime', 1), ('using', 1),
('typing.get_type_hints().', 1), ('In', 1), ('common', 1), ('case', 1),
('where', 1), ('this', 1), ('is', 1), ('not', 1), ('required,', 1), ('cheaper',
1), ('store', 1), ('(since', 1), ('short', 1), ('strings', 1), ('interned', 1),
('interpreter)', 1), ('and', 1), ('make', 1), ('startup', 1), ('time', 1),
('faster.', 1), ('by', 2), ('at', 2), ('to', 2), ('are', 3), ('in', 3),
('annotations', 3), ('of', 4), ('the', 8)]
('of', 4)
```

Vishal Acharya

Vishal Acharya

10 Write a Python program to create a string from two given strings concatenating uncommon characters of the said strings

[16]:
```
def uncommon_chars_concat(s1, s2):

    set1 = set(s1)
    set2 = set(s2)

    common_chars = list(set1 & set2)
    result = [ch for ch in s1 if ch not in common_chars] + [ch for ch in s2 if␣
 ↪ch not in common_chars]
    return(''.join(result))

s1 = 'abcdpqr'
s2 = 'xyzabcd'
print("Original Substrings:\n",s1+"\n",s2)
print("\nAfter concatenating uncommon characters:")
print(uncommon_chars_concat(s1, s2))
```

```
Original Substrings:
 abcdpqr
 xyzabcd

After concatenating uncommon characters:
pqrxyz
```

11 Write a Python program to find the minimum window in a given string which will contain all the characters of another given string. (You can use your own method to find the solution as well)

Input : str1 = ” PRWSOERIUSFK ”

str2 = ” OSU ”

Output: Minimum window is “OERIUS

[25]:
```
def minWindow(s,t) :

        counter_t = {}
        counter_s = {}

        for c in t:
            counter_t[c] = counter_t.get(c, 0) + 1


        i = 0
```

Vishal Acharya

```
        j = 0

        left = -1
        right = -1

        valid = 0

        for i in range(len(s)):

            while j < len(s) and valid < len(counter_t):
                counter_s[s[j]] = counter_s.get(s[j], 0) + 1

                if s[j] in counter_t and counter_s[s[j]] == counter_t[s[j]]:
                    valid += 1


                j += 1


            if valid == len(counter_t):
                if left == -1 or j - i < right - left:
                    left = i
                    right = j


            counter_s[s[i]] -= 1
            if s[i] in counter_t and counter_s[s[i]] == counter_t[s[i]] - 1:
                valid -= 1

        if left == -1:
            return "not found"


        return s[left : right]
s="1a2b3cbva"
t="abc"
print(minWindow(s,t))
s = "PRWSvOERIUSFK"
t = "OSU"
print(minWindow(s,t))
s="abcfghioi9i99i9i9ijnbhhjkmijnbhfi  hklij %^^hilkiujj␣
 ↪hjmjkfrtygh000000000ijnhjhijhhgfghjmnbvghoiuhghgfij hjkjij"
t="hijkwxy"
print(minWindow(s,t))
```

```
cbva
OERIUS
not found
```

Vishal Acharya

[1]:
```
def password_check(password):
    l, u, p, d = 0, 0, 0, 0
    if (len(password) >= 8 and len(password) <= 15):
        for i in password:
        # counting lowercase alphabets
            if (i.islower()):
                l+=1
        # counting uppercase alphabets
            if (i.isupper()):
                u+=1
        # counting digits
            if (i.isdigit()):
                d+=1
        # counting the mentioned special characters
            if(i=='@'or i=='$' or i=='_'):
                p+=1
        if (l>=1 and u>=1 and p>=1 and d>=1 and l+p+u+d==len(password)):
            return True
        else:
            return False
    else:
        return False
d={"student0":'Student@0',"student1":'Student@11',"student2":
 ↪'Student@121',"student3":'Student@052',"student4":'Student@01278',
  "student5":'Student@0125',   "student6":'Student@042',"student7":
 ↪'Student@07800',"student8":'Student@012',
  "student9":'Student@04789'}
first_password_username_count=0
s="  "
while first_password_username_count<3:
    if s=="stop":
        break
    username=input("enter correct username:")
    password=input("enter correct password:")
    if ((username not in d.keys()) or (d[username]!=password)):
        print("enter correct username and password")
        first_password_username_count+=1
        continue
    else:
        #update password
        update_count=0
        while(update_count<3):
            update_password=input("enter update password: ")
            if password_check(update_password):
                d[username]=update_password
                s="stop"
                break
```

Vishal Acharya

```
            else:
                update_count+=1
                print(""" # The Password must have:
         1. at least 1 number between 0 and 9
         2. at least 1 upper letter (between a and z)
         3. at least 1 lower letter (between A and Z)
         4. at least 1 special character out of @$_:
         5. minimum length of password is 8 and maximum length is 15""")
                continue
        else:
            print("try after 24h")
            s="stop"
            break
else:
    print("try after 24h")

print(d)
#Find and print longest and shortest password with its username
sorted_list=sorted(d.items(),key=lambda x:len(x[1]))
print("longest password",sorted_list[-1][0],"--",sorted_list[-1][1])
print("shortest password",sorted_list[0][0],"--",sorted_list[0][1])
```

```
enter correct username:student0
enter correct password:Student@0
enter update password: Student@88888
{'student0': 'Student@88888', 'student1': 'Student@11', 'student2':
'Student@121', 'student3': 'Student@052', 'student4': 'Student@01278',
'student5': 'Student@0125', 'student6': 'Student@042', 'student7':
'Student@07800', 'student8': 'Student@012', 'student9': 'Student@04789'}
longest password student9 -- Student@04789
shortest password student1 -- Student@11
```

12 Write a Python program to remove unwanted characters from a given string.

```
Sample Output:

    Original String : Pyth*^on Exercis^es

        After removing unwanted characters:

            Python Exercises

            Original String : A%^!B#*CD

After removing unwanted characters:
```

```
ABCD
```

[ ]:

[32]:
```
def remove_chars(str1, unwanted_chars):
    for i in unwanted_chars:
        str1 = str1.replace(i, '')
    return str1



str1 = "Pyth*^on Exercis^es"
str2 = "A%^!B#*CD"

unwanted_chars = ["#", "*", "!", "^", "%"]
print ("Original String : " + str1)
print("After removing unwanted characters:")
print(remove_chars(str1, unwanted_chars))
print ("\nOriginal String : " + str2)
print("After removing unwanted characters:")
print(remove_chars(str2, unwanted_chars))
```

```
Original String : Pyth*^on Exercis^es
After removing unwanted characters:
Python Exercises

Original String : A%^!B#*CD
After removing unwanted characters:
ABCD
```

13 Write a Python program to remove punctuations from a given string.

Sample Output:

Original text:

String! With. Punctuation?

After removing Punctuations from the said string:

String With Punctuation

[33]:
```
def remove_punctuations(text):
    punc_list = '''!()-[]{};:'"\,<>./?@#$%^&*_~'''
    result = ""
    for char in text:
        if char not in punc_list:
            result = result + char
```

Vishal Acharya

```
    return result
text = "@^&$String! With.-- Punctuation?"
print("Original text:")
print(text)
result = remove_punctuations(text)
print("\nAfter removing Punctuations from the said string:")
print(result)
```

```
Original text:
@^&$String! With.-- Punctuation?

After removing Punctuations from the said string:
String With Punctuation
```

14 Write a Python program to remove repeated consecutive characters and replace with the single letters and print new updated string.

Sample Data:

("Red Green White") -> "Red Gren White"

("aabbbcdeffff") -> "abcdef"

("Yellowwooddoor") -> "Yelowodor"

[34]:
```
def test(text):
  result = []
  for x in text:
    if not result or result[-1] != x:
      result.append(x)
  return ''.join(result)

text ="Red Green White"
print("Original string:", text)
print("Remove repeated consecutive characters and replace with the single␣
 ↪letters:")
print(test(text))
text = "aabbbcdeffff"
print("\nOriginal string:", text)
print("Remove repeated consecutive characters and replace with the single␣
 ↪letters:")
print(test(text))
text = "Yellowwooddoor"
print("\nOriginal string:", text)
print("Remove repeated consecutive characters and replace with the single␣
 ↪letters:")
print(test(text))
```

Vishal Acharya

```
Original string: Red Green White
Remove repeated consecutive characters and replace with the single letters:
Red Gren White

Original string: aabbbcdeffff
Remove repeated consecutive characters and replace with the single letters:
abcdef

Original string: Yellowwooddoor
Remove repeated consecutive characters and replace with the single letters:
Yelowodor
```

[35]:
```
def test(text):
    return ''.join(text[i] for i in range(len(text)) if i==0 or text[i-1]!
 ↪=text[i])
text ="Red Green White"
print("Original string:", text)
print("Remove repeated consecutive characters and replace with the single␣
 ↪letters:")
print(test(text))
text = "aabbbcdeffff"
print("\nOriginal string:", text)
print("Remove repeated consecutive characters and replace with the single␣
 ↪letters:")
print(test(text))
text = "Yellowwooddoor"
print("\nOriginal string:", text)
print("Remove repeated consecutive characters and replace with the single␣
 ↪letters:")
print(test(text))
```

```
Original string: Red Green White
Remove repeated consecutive characters and replace with the single letters:
Red Gren White

Original string: aabbbcdeffff
Remove repeated consecutive characters and replace with the single letters:
abcdef

Original string: Yellowwooddoor
Remove repeated consecutive characters and replace with the single letters:
Yelowodor
```

Vishal Acharya

15 Write a Python program to that takes two strings. Count the number of times each string contains the same three letters at the same index. Go to the editor

Sample Data:

("Red RedGreen") -> 1

("Red White Red White) -> 7

("Red White White Red") -> 0

[36]:
```
def test(text1, text2):
            ctr = 0
            for i in range(len(text1) - 2):
                        if text1[i:i+3] == text2[i:i+3]:
                                    ctr += 1
            return ctr
text1 ="Red"
text2 ="RedGreen"
print("Original strings:", text1,text2)
print("Check said two strings contain three letters at the same index:")
print(test(text1, text2))
text1 ="Red White"
text2 ="Red White"
print("Original strings:", text1,text2)
print("Check said two strings contain three letters at the same index:")
print(test(text1, text2))
text1 ="Red White"
text2 ="White Red"
print("Original strings:", text1,text2)
print("Check said two strings contain three letters at the same index:")
print(test(text1, text2))
```

```
Original strings: Red RedGreen
Check said two strings contain three letters at the same index:
1
Original strings: Red White Red White
Check said two strings contain three letters at the same index:
7
Original strings: Red White White Red
Check said two strings contain three letters at the same index:
0
```

Vishal Acharya

16 Write a Python script to sort (ascending and descending) a dictionary by value.

[37]:
```
def sort_dict_by_value(d, reverse = False):
  return dict(sorted(d.items(), key = lambda x: x[1], reverse = reverse))
print("Original dictionary elements:")
colors = {'Red': 1, 'Green': 3, 'Black': 5, 'White': 2, 'Pink': 4}
print(colors)
print("\nSort (ascending) the said dictionary elements by value:")
print(sort_dict_by_value(colors))
print("\nSort (descending) the said dictionary elements by value:")
print(sort_dict_by_value(colors, True))
```

```
Original dictionary elements:
{'Red': 1, 'Green': 3, 'Black': 5, 'White': 2, 'Pink': 4}

Sort (ascending) the said dictionary elements by value:
{'Red': 1, 'White': 2, 'Green': 3, 'Pink': 4, 'Black': 5}

Sort (descending) the said dictionary elements by value:
{'Black': 5, 'Pink': 4, 'Green': 3, 'White': 2, 'Red': 1}
```

17 I have a string of words. I would like to sort the list by the length of each word so that the longest word is at the top

[1]:
```
s="nbhg mjknh kiuytgft nbh nbhgtygnm mnjku lkmnhg nbh nb ghyuytredf bghyyt␣
  ↪kmnjkmnjkmn bghbghb vfgbvc hgv bvg"
d={}
for i in s.split():
    d[i]=len(i)
#print(d)
l1=d.values()
l2=d.keys()
#print(l1)
#print(l2)
ans=[i[1] for i in sorted(list(zip(l1,l2)),reverse=True)]
print(ans)
```

```
['kmnjkmnjkmn', 'ghyuytredf', 'nbhgtygnm', 'kiuytgft', 'bghbghb', 'vfgbvc',
'lkmnhg', 'bghyyt', 'mnjku', 'mjknh', 'nbhg', 'nbh', 'hgv', 'bvg', 'nb']
```

Vishal Acharya

18 This is a Python Program to create a dictionary with key as first character and value as words starting with that character.

[ ]:
```
s="acd bcd avf ght dfg bgh kju avf ghv bvf ujh kju bgh avf nhj bgh bvf cdf avg␣
 ↪sde kju gty frt der cdf xsd zse"
d={}
for i in s.split():
    if i[0] not in d.keys():
        d[i[0]]=[]
        d[i[0]].append(i)
    else:
        d[i[0]].append(i)
print(d)
```

19 This is a Python Program to create a dictionary with key as character and value as character count repeated in string.first five more repeated character

[39]:
```
s="acd bcd avf ght dfg bgh kju avf ghv bvf ujh kju bgh avf nhj bgh bvf cdf avg␣
 ↪sde kju gty frt der cdf xsd zse"
d={}
for i in s.split():
    d[i[0]]=d.get(i[0],0)+1
print(d)
y=sorted(d.items(),key=lambda x:x[1],reverse=True)
for i in range(5):
    print(y[i])
```

```
{'a': 5, 'b': 6, 'g': 3, 'd': 2, 'k': 3, 'u': 1, 'n': 1, 'c': 2, 's': 1, 'f': 1,
'x': 1, 'z': 1}
('b', 6)
('a', 5)
('g', 3)
('k', 3)
('d', 2)
```

20 Sort Dictionary key and values List.

Example 1:

Input:

{‘c’: [3], ‘b’: [12, 10], ‘a’: [19, 4]}

Output:

{‘a’: [4, 19], ‘b’: [10, 12], ‘c’: [3]}

Vishal Acharya

Example 2:

Input:

{‘c’: [10, 34, 3]}

Output:

{‘c’: [3, 10, 34]}

[40]:
```
d = {'c': [3], 'b': [12, 10], 'a': [19, 4]}

res = {}

for i in sorted(d):
  res[i] = sorted(d[i])

print(res)
```

```
{'a': [4, 19], 'b': [10, 12], 'c': [3]}
```

21 Convert a list of Tuples into Dictionary.

Example 1:

Input:

[(“akash”, 10), (“gaurav”, 12), (“anand”, 14), (“suraj”, 20), (“akhil”, 25), (“ashish”, 30)]

Output:

{‘akash’: [10], ‘gaurav’: [12], ‘anand’: [14], ‘suraj’: [20], ‘akhil’: [25], ‘ashish’: [30]}

Example 2:

Input:

[(‘A’, 1), (‘B’, 2), (‘C’, 3)]

Output:

{‘A’: [1], ‘B’: [2], ‘C’: [3]}

[41]:
```
L1 = [("akash", 10), ("gaurav", 12), ("anand", 14), ("suraj", 20), ("akhil",␣
  ↪25), ("ashish", 30)]

L = [('A', 1), ('B', 2), ('C', 3)]

d = {}

for i,j in L:
  d[i] = [j]

print(d)
```

Vishal Acharya

```
{'A': [1], 'B': [2], 'C': [3]}
```

22 Key with maximum unique values

Given a dictionary with values list, extract key whose value has most unique values.

Example 1:

Input:

test_dict = {“CampusX” : [5, 7, 9, 4, 0], “is” : [6, 7, 4, 3, 3], “Best” : [9, 9, 6, 5, 5]}

Output:

CampusX

Example 2:

Input:

test_dict = {“CampusX” : [5, 7, 7, 7, 7], “is” : [6, 7, 7, 7], “Best” : [9, 9, 6, 5, 5]}

Output:

Best

[42]:
```
test_dict = {"CampusX" : [5, 7, 7, 7, 7], "is" : [6, 7, 7, 7], "Best" : [9, 9,␣
 ↪6, 5, 5]}

max_val = 0
max_key = ''
for i in test_dict:
  if max_val < len(set(test_dict[i])):
    max_val = len(set(test_dict[i]))
    max_key = i

print(max_key)
```

```
Best
```

23 find uncommon words from two Strings. Statement: Given two sentences as strings A and B. The task is to return a list of all uncommon words. A word is uncommon if it appears exactly once in any one of the sentences, and does not appear in the other sentence. Note: A sentence is a string of space-separated words. Each word consists only of lowercase letters.

Example 1:

Input:

A = “apple banana mango” B = “banana fruits mango”

Vishal Acharya

Vishal Acharya

[3]:
```
# Code here
A = "apple banana mango"
B = "banana fruits mango"

L = []

for i in A.split():
  if i not in B and i not in L:
    L.append(i)

for j in B.split():
  if j not in A and j not in L:
    L.append(j)

print(L)
```

```
['apple', 'fruits']
```

24 Check whether the string is Symmetrical. Statement: Given a string. the task is to check if the string is symmetrical or not. A string is said to be symmetrical if both the halves of the string are the same.

Example 1:

Input

khokho Output

The entered string is symmetrical

[5]:
```
s = input('enter the string')

if len(s)%2 == 0:
  s1 = s[0:len(s)//2]
  s2 = s[len(s)//2:]
else:
  s1 = s[0:len(s)//2]
  s2 = s[len(s)//2 + 1:]
if s1 == s2:
  print('symmetrical')
else:
  print('not symmetrical')
```

```
enter the stringKHOKHO
symmetrical
```

Vishal Acharya

25 Take a alphanumeric string input and print the sum and average of the digits that appear in the string, ignoring all other characters. Input:

hel123O4every093

Output:

Sum: 22 Avg: 3.14

[6]:
```
s = 'hel123O4every093'

sum = 0
count = 0

for i in s:
  if i.isdigit():
    sum = sum + int(i)
    count += 1

print(sum)
print(sum/count)
print(count)
```

```
22
3.142857142857143
7
```

26 Create Short Form from initial character

Given a string create short form ofthe string from Initial character. Short form should be capitalised.

Example:

Input:

Data science mentorship program Output:

DSMP

[7]:
```
inp = 'Data science Mentorship Program'

res = ''

for i in inp.split():
  res = res + i[0].upper()

print(res)
```

```
DSMP
```

Vishal Acharya

26.0.1 Given a list L of size N. You need to count the number of special elements in the given list.

26.0.2 An element is special if removal of that element makes the list balanced. The list will be balanced if sum of even index elements is equal to the sum of odd index elements.

Example Input

Input 1:

A = [2, 1, 6, 4] Input 2:

A = [5, 5, 2, 5, 8] Example Output

Output 1:

1

Output 2:

2

Explanation 1:

After deleting 1 from list : [2,6,4] (2+4) = (6)

Hence 1 is the only special element, so count is 1 Explanation 2:

If we delete A[0] or A[1] , list will be balanced (5+5) = (2+8)

So A[0] and A[1] are special elements, so count is 2.

[1]:
```
l=eval(input("enter list"))
count=0
for i in range(0,len(l)):
    c=l.copy()
    c.pop(i)
    sum1=sum(c[0::2])
    sum2=sum(c[1::2])
    if sum1==sum2:
        print(i)
        count+=1
print("total count",count)
```

```
enter list[1,2,3,2]
1
total count 1
```

Vishal Acharya

26.0.3 **A simple way of encrypting a message is to rearrange its characters. One way to rearrange the characters is to pick out the characters at even indices, put them first in the encrypted string, and follow them by the odd characters. For example, the string message would be encrypted as msaeesg because the even characters are m, s, a, e (at indices 0, 2, 4, and 6) and the odd characters are e, s, g (at indices 1, 3, and 5).**

26.0.4 **(1). Write a program that asks the user for a string and uses this method to encrypt the string.**

26.0.5 **(2). Write a program that decrypts a string that was encrypted with this method.**

[6]:
```
def encrypt():#to put all our code in function
    strings=input('what is your message: ')
    even=strings[::2]#extract the even part
    odd=strings[1::2]#extract the odd part
    print(even+odd)#print the result
encrypt()

def decrypt():
    string=input('what is your message: ')
    length=len(string)#to get the length of the input
    half_length=(length+1)//2 #half of the input
    even=string[:half_length]#even part
    odd=string[half_length:]#odd part
#here we start inserting each of the part to form original
    msg=''
    for i in range (half_length):
        join=even[i:i+1]+odd[i:i+1]
        msg=msg+join
    print(msg)
decrypt()
```

```
what is your message: This is python programming
Ti spto rgamnhsi yhnpormig
what is your message: Ti spto rgamnhsi yhnpormig
This is python programming
```

Vishal Acharya

27 A more general version of the above technique is the rail fence cipher, where instead of breaking things into evens and odds, they are broken up by threes, fours or something larger. For instance, in the case of threes, the string secret message would be broken into three groups. The first group is sr sg, the characters at indices 0, 3, 6, 9 and 12. The second group is eemse, the characters at indices 1, 4, 7, 10, and 13. The last group is ctea, the characters at indices 2, 5, 8, and 11. The encrypted message is sr sgeemsectea.

27.0.1 Write a program that asks the user for a string, and an integer determining whether to break things up by threes, fours, or whatever user inputs. Encrypt the string using above method.

27.0.2 Write a decryption program for the same general case. Taking input of any encrypted string from user with number used to break things apart during encryption.

27.0.3 Encryption

[7]:
```
s=input()
s1=""
key=int(input())
for i in range(key):
    s1+=s[i::key]
print(s1)
```

```
This is python, a programming language
5
Tit omlahshagiagi o rnnespnpagg y,rm u
```

27.0.4 1st Solution for Decryption

[8]:
```
string=input('what is your message: ')
key=int(input())
length=len(string)#to get the length of the input
part=length//key #half of the input
extra=length%key #extra characters after dividing string in equal parts
part1=string[:(part+1)*extra]
part2=string[(part+1)*extra:]
msg=''
for i in range(part+1):
    if i<part:
        msg+=part1[i::part+1]+part2[i::part]
    else:
        msg+=part1[i::part+1]
```

```
print(msg)
```

```
what is your message: Tit omlahshagiagi o rnnespnpagg y,rm u
5
This is python, a programming language
```

27.0.5 2nd Solution for Decryption

[9]:
```
string=input('what is your message: ')
key=int(input())
length=len(string)#to get the length of the input
part=length//key #half of the input
extra=length%key #extra characters after dividing string in equal parts
part1=string[:(part+1)*extra]
part2=string[(part+1)*extra:]
msg=''
for i in range(part+1):
    if i<part:
        msg+=part1[i::part+1]+part2[i::part]
    else:
        msg+=part1[i::part+1]
print(msg)
```

```
what is your message: Tit omlahshagiagi o rnnespnpagg y,rm u
5
This is python, a programming language
```

28 Write a Python program which will return the sum of the numbers in the array, returning 0 for an empty array. Except the number 13 is very unlucky, so it does not count and number that come immediately after 13 also do not count.

Example : [1, 2, 3, 4] = 10 [1, 2, 3, 4, 13] = 10 [13, 1, 2, 3, 13] = 5

[5]:
```
def sum_list(l):
    if len(l)==0:
        return 0
    else:
        sum=0
        for i in range(len(l)):
            if l[i]==13 or l[i-1]==13 and i!=0:
                continue
            else:
                sum+=l[i]
        return sum
l=eval(input("enter list"))
print(sum_list(l))
```

Vishal Acharya

```
enter list[1,13,1,2,13,3,3,13]
6
```

29 Write Python Program to Add Two Matrices

matrix_1 = [[1, 2, 3],
    [4, 5, 6],

    [7, 8, 9]]

matrix_2 = [[1, 2, 3],
    [4, 5, 6],

    [7, 8, 9]]

[7]:
```
matrix_1 = [[1, 2, 3],[4, 5, 6], [7, 8, 9]]
matrix_2 = [[1, 2, 3],[4, 5, 6], [7, 8, 9]]
matrix_result = [[0, 0, 0],[0, 0, 0],[0, 0, 0]]
for rows in range(len(matrix_1)):
    for columns in range(len(matrix_2[0])):
        matrix_result[rows][columns] = matrix_1[rows][columns] +␣
 ↪matrix_2[rows][columns]
print("Addition of two matrices is")
print(matrix_result)
for items in matrix_result:
    print(items)
```

```
Addition of two matrices is
[[2, 4, 6], [8, 10, 12], [14, 16, 18]]
[2, 4, 6]
[8, 10, 12]
[14, 16, 18]
```

30 Program to multiply two matrices using nested for loops

[8]:
```
#Program to multiply two matrices using nested for loops
# 3x3 matrix
A = [[1,2,3],
     [4,5,6],
     [7,8,9]]

B = [[1,2,3,4],
     [5,6,7,8],
     [2,4,6,8]]

result = [[0,0,0,0],
```

Vishal Acharya

```
          [0,0,0,0],
          [0,0,0,0]]


for i in range(len(A)):

    for j in range(len(B[0])):

        for k in range(len(B)):
            result[i][j] += A[i][k] * B[k][j]
print(result)
print('Multiplied Matrix:')
for r in result:
    print(r)
```

```
[[17, 26, 35, 44], [41, 62, 83, 104], [65, 98, 131, 164]]
Multiplied Matrix:
[17, 26, 35, 44]
[41, 62, 83, 104]
[65, 98, 131, 164]
```

[9]:
```
#make the dictionary as key is charcter and value is the count of character in␣
 ↪string.after print
#first five max repeated character
s="avbgkbhjdyfbckbvdfgbybncvhbvhcvbmnbvcxasdfghjklpoiuytrewqhjujnbvawdpscmsbndysnvnbvbnfddnbjv
d={}
for i in s:
    d[i]=d.get(i,0)+1
#print(d)
k=sorted(d.items(),key=lambda x:x[1],reverse=True )
#print(k)
print("first five max repeated character")
for i in range(0,5):
    print(k[i])
```

```
first five max repeated character
('b', 14)
('v', 10)
('n', 8)
('d', 7)
('h', 5)
```

Vishal Acharya

Vishal Acharya

31 Q. Repeatedly ask the user to enter a team name and how many games the team has won and how many they lost.Store this information in a dictionary where the keys are the team names and the values are a list of the form [wins, losses].

(i) using the dictionary created above, allow the user to enter a team name and print out the team's winning percentage.

(ii) using dictionary create a list whose entries are the number of wins for each team.

(iii) using the dictionary, create a list of all those teams that have winning records.

[10]:
```
dic ={}
lstwin = []
lstrec = []
while True :
    name = input ("Enter the name of team (enter q for quit)= ")
    if name == "Q" or name == "q" :
        print()
        break
    else :
        win = int (input("Enter the no.of win match = "))
        loss = int(input("Enter the no.of loss match = "))
        print()
        dic [ name ] = [ win , loss ]
        lstwin += [ win ]
        if win > 0 :
            lstrec += [ name ]

nam = input ("Enter the name of team For Winning = ")
print ("Winning percentage = ",dic [ nam ][0] *100 / (dic [nam ][0] + dic[nam␣
↪][1] ))
print()
print("Winning list of all team = ",lstwin)
print("Team who has winning records are ",lstrec)
```

```
Enter the name of team (enter q for quit)= nhm
Enter the no.of win match = 8
Enter the no.of loss match = 2

Enter the name of team (enter q for quit)= nhj
Enter the no.of win match = 7
Enter the no.of loss match = 5

Enter the name of team (enter q for quit)= q

Enter the name of team For Winning = nhm
Winning percentage =  80.0
```

```
Winning list of all team =  [8, 7]
Team who has winning records are  ['nhm', 'nhj']
```

32 Write a Python function that accepts a string and calculate the number of upper case letters and lower case letters.

[11]:
```
#Write a Python function that accepts a string and calculate the number of␣
 ↪upper case letters and lower case letters.
def up_low(string):
    uppers = 0
    lowers = 0
    for char in string:
        if char.islower():
            lowers += 1
        elif char.isupper():
            uppers +=1

    return(uppers, lowers)

print(up_low('Hello Mr. Rogers, how are you this fine Tuesday?'))
```

```
(4, 33)
```

33 Use a list comprehension to produce a list that consists of all palindromic numbers between 100 and 1000.

[12]:
```
L=[i for i in range(100,1001) if str(i)==str(i)[::-1]]
print(L)
```

```
[101, 111, 121, 131, 141, 151, 161, 171, 181, 191, 202, 212, 222, 232, 242, 252,
262, 272, 282, 292, 303, 313, 323, 333, 343, 353, 363, 373, 383, 393, 404, 414,
424, 434, 444, 454, 464, 474, 484, 494, 505, 515, 525, 535, 545, 555, 565, 575,
585, 595, 606, 616, 626, 636, 646, 656, 666, 676, 686, 696, 707, 717, 727, 737,
747, 757, 767, 777, 787, 797, 808, 818, 828, 838, 848, 858, 868, 878, 888, 898,
909, 919, 929, 939, 949, 959, 969, 979, 989, 999]
```

Vishal Acharya

34 **Write a program that converts Roman numerals into ordinary numbers. Here are the conversions: M=1000, D=500, C=100, L=50, X=10, V=5 I=1. Don't forget about things like IV being 4 and XL being 40.**

35 **Write a program that converts ordinary numbers into Roman numerals**

[1]:
```
def romanToInt(s):
        """
        :type s: str
        :rtype: int
        """
        roman = {'I':1,'V':5,'X':10,'L':50,'C':100,'D':500,'M':1000,'IV':4,'IX':
↪9,'XL':40,'XC':90,'CD':400,'CM':900}
        i = 0
        num = 0
        while i < len(s):
            if i+1<len(s) and s[i:i+2] in roman:
                num+=roman[s[i:i+2]]
                i+=2
            else:
                num+=roman[s[i]]
                i+=1
        return num
s=input("Enter number in Roman: ")
print(romanToInt(s))
```

```
Enter number in Roman: LVIII
58
```

[14]:
```
def int_to_Roman(num):
        val = [
            1000, 900, 500, 400,
            100, 90, 50, 40,
            10, 9, 5, 4,
            1
            ]
        syb = [
            "M", "CM", "D", "CD",
            "C", "XC", "L", "XL",
            "X", "IX", "V", "IV",
            "I"
            ]
        roman_num = ''
        i = 0
```

Vishal Acharya

```
        while  num > 0:
            for j in range(num // val[i]):
                roman_num += syb[i]
                num -= val[i]
            i += 1
        return roman_num
num=int(input("Input an integer: "))
print(int_to_Roman(num))
```

```
Input an integer: 58
LVIII
```

36 Create a 5 × 5 list of numbers. Then write a program that creates a dictionary whose keys are the numbers and whose values are the how many times the number occurs. Then print the three most common numbers.

[15]:
```
L=[[5, 3, 3, 5, 5],
   [3, 2, 4, 3, 3],
   [3, 3, 3, 3, 4],
   [3, 4, 5, 3, 3],
   [5, 4, 1, 2, 3]]
d={}
for i in range(len(L)):
    for j in L[i]:
        d[j]=d.get(j,0)+1
print(d)
L1=[]
for i,j in d.items():
    L1.append([j,i])
L1.sort()
L1.reverse()
L2=[]
for i in range(3):
    print(L1[i][1],'comes',L1[i][0],'times in a 5x5 list')
    L2.append(L1[i][1])
print('Three most common numbers in 5x5 list are', L2)
```

```
{5: 5, 3: 13, 2: 2, 4: 4, 1: 1}
3 comes 13 times in a 5x5 list
5 comes 5 times in a 5x5 list
4 comes 4 times in a 5x5 list
Three most common numbers in 5x5 list are [3, 5, 4]
```

Vishal Acharya

Vishal Acharya

37 Given an integer value, return a string with the equivalent English text of each digit. For example, an input of 89 results in "eighty-nine" being returned. Restrict values to be between 0 and 1,000.

[2]:
```
"""Given an int32 number, print it in English."""
def int_to_en(num):
    d = { 0 : 'zero', 1 : 'one', 2 : 'two', 3 : 'three', 4 : 'four', 5 : 'five',
          6 : 'six', 7 : 'seven', 8 : 'eight', 9 : 'nine', 10 : 'ten',
          11 : 'eleven', 12 : 'twelve', 13 : 'thirteen', 14 : 'fourteen',
          15 : 'fifteen', 16 : 'sixteen', 17 : 'seventeen', 18 : 'eighteen',
          19 : 'nineteen', 20 : 'twenty',
          30 : 'thirty', 40 : 'forty', 50 : 'fifty', 60 : 'sixty',
          70 : 'seventy', 80 : 'eighty', 90 : 'ninety' }
    k = 1000

    assert(0 <= num)

    if (num < 20):
        return d[num]

    if (num < 100):
        if num % 10 == 0: return d[num]
        else: return d[num // 10 * 10] + '-' + d[num % 10]

    if (num < k):
        if num % 100 == 0: return d[num // 100] + ' hundred'
        else: return d[num // 100] + ' hundred and ' + int_to_en(num % 100)

print(int_to_en(200))
```

```
two hundred
```

38 Given an integer value, return a string with the equivalent English text of each digit. For example, an input of 89 results in "eighty-nine" being returned. Restrict values to be between 0 and 10^15.

[16]:
```
def int_to_en(num):
    d = { 0 : 'zero', 1 : 'one', 2 : 'two', 3 : 'three', 4 : 'four', 5 : 'five',
          6 : 'six', 7 : 'seven', 8 : 'eight', 9 : 'nine', 10 : 'ten',
          11 : 'eleven', 12 : 'twelve', 13 : 'thirteen', 14 : 'fourteen',
          15 : 'fifteen', 16 : 'sixteen', 17 : 'seventeen', 18 : 'eighteen',
          19 : 'nineteen', 20 : 'twenty',
          30 : 'thirty', 40 : 'forty', 50 : 'fifty', 60 : 'sixty',
```

Vishal Acharya

```
          70 : 'seventy', 80 : 'eighty', 90 : 'ninety' }
    k = 1000
    m = k * 1000
    b = m * 1000
    t = b * 1000

    if (num < 20):
        return d[num]

    if (num < 100):
        if num % 10 == 0:
            return d[num]
        else:
            return d[num // 10 * 10] + '-' + d[num % 10]

    if (num < k):
        if num % 100 == 0:
            return d[num // 100] + ' hundred'
        else:
            return d[num // 100] + ' hundred and ' + int_to_en(num % 100)

    if (num < m):
        if num % k == 0:
            return int_to_en(num // k) + ' thousand'
        else:
            return int_to_en(num // k) + ' thousand, ' + int_to_en(num % k)

    if (num < b):
        if (num % m) == 0:
            return int_to_en(num // m) + ' million'
        else:
            return int_to_en(num // m) + ' million, ' + int_to_en(num % m)

    if (num < t):
        if (num % b) == 0:
            return int_to_en(num // b) + ' billion'
        else:
            return int_to_en(num // b) + ' billion, ' + int_to_en(num % b)

    if (num >= t):
        if (num % t == 0):
            return int_to_en(num // t) + ' trillion'
        else:
            return int_to_en(num // t) + ' trillion, ' + int_to_en(num % t)
num=int(input("Enter any positive integer: "))
print(int_to_en(num))
```

```
Enter any positive integer: 3669478521369427
```

three thousand, six hundred and sixty-nine trillion, four hundred and seventy-eight billion, five hundred and twenty-one million, three hundred and sixty-nine thousand, four hundred and twenty-seven

38.1 Write a Python program to count the number ofstrings where the string length is 2 or more and the first and last character are same from a given list of strings.

Example: Input: [‘abc’, ‘xyz’, ‘aba’, ‘1221’]

Output: 2

[10]:
```
def match_words(words):
  ctr = 0

  for word in words:
    if len(word) > 1 and word[0] == word[-1]:
      ctr += 1
  return ctr

print(match_words(['abc', 'xyz', 'aba', '1221']))
```

2

38.2 Find the indices of all occurrences of target in the uneven matrix

38.3 An irregular/uneven matrix, or ragged matrix, is a matrix that has a different number of elements in each row. Ragged matrices are not used in linear algebra, since standard matrix transformations cannot be performed on them, but they are useful as arrays in computing.

38.4 Write a Python program to find the indices of all occurrences of target in the uneven matrix.

Input: [[1, 3, 2, 32, 19], [19, 2, 48, 19], [], [9, 35, 4], [3, 19], 19]

Output: [[0, 4], [1, 0], [1, 3], [4, 1]]

Input: [[1, 2, 3, 2], [], [7, 9, 2, 1, 4],2]

Output: [[0, 1], [0, 3], [2, 2]]

[14]:
```
def test(M, T):
    L=[]
    for i,row in enumerate(M):
        for j,k in enumerate(row):
            if k==T:
                L.append([i,j])
    return L
M = [[1, 3, 2, 32, 19], [19, 2, 48, 19], [], [9, 35, 4], [3, 19]]
T = 19
```

Vishal Acharya

```
print("Matrix:")
print(M)
print("Target value:")
print(T)
print("Indices of all occurrences of the target value in the said uneven matrix:
 ↪")
print(test(M,T))
```

```
Matrix:
[[1, 3, 2, 32, 19], [19, 2, 48, 19], [], [9, 35, 4], [3, 19]]
Target value:
19
Indices of all occurrences of the target value in the said uneven matrix:
[[0, 4], [1, 0], [1, 3], [4, 1]]
```

38.5 Write a Python program to find the numbers that are greater than 10 and have odd first and last digits.

Input: [1, 3, 79, 10, 4, 1, 39, 62]

Output: [79, 39]

Input: [11, 31, 77, 93, 48, 1, 57]

Output: [11, 31, 77, 93, 57]

[19]:
```
def test(nums):
    L=[]
    for i in nums:
        if i>10:
            if int(str(i)[len(str(i))-1])%2==1 and int(str(i)[0]) % 2==1:
                L.append(i)
    return L

nums = [1, 3, 79, 10, 4, 1, 39]
print("Original list of numbers:")
print(nums)
print("Numbers of the said array that are greater than 10 and have odd first␣
 ↪and last digits:")
print(test(nums))
```

```
Original list of numbers:
[1, 3, 79, 10, 4, 1, 39]
Numbers of the said array that are greater than 10 and have odd first and last
digits:
[79, 39]
```

Vishal Acharya

38.6 Write a Python program to shift the decimal digits n places to the left, wrapping the extra digits around. If shift > the number of digits of n, reverse the string.

[21]:
```
def test(n, shift):
    s = str(n)
    if shift > len(s):
        return s[::-1]
    return s[shift:] + s[:shift]

print("Shift the decimal digits n places to the left. If shift > the number of␣
 ↪digits of n, reverse the string.:")

n = 12345
shift = 1
print("\nn =",n," and shift =",shift)
print("Result = ",test(n, shift))
n = 12345
shift = 2
print("\nn =",n," and shift =",shift)
print("Result = ",test(n, shift))
n = 12345
shift = 3
print("\nn =",n," and shift =",shift)
print("Result = ",test(n, shift))
n = 12345
shift = 5
print("\nn =",n," and shift =",shift)
print("Result = ",test(n, shift))
n = 12345
shift = 7
print("\nn =",n," and shift =",shift)
print("Result = ",test(n, shift))
```

```
Shift the decimal digits n places to the left. If shift > the number of digits
of n, reverse the string.:

n = 12345  and shift = 1
Result =  23451

n = 12345  and shift = 2
Result =  34512

n = 12345  and shift = 3
Result =  45123

n = 12345  and shift = 5
Result =  12345
```

Vishal Acharya

```
n = 12345  and shift = 7
Result =  54321
```

38.7 write a Python program to create a list containing that number in between each pair of adjacent numbers.

Input: [12, -7, 3, -89, 14, 88, -78, -1, 2, 7]

Separator: 6

Output: [12, 6, -7, 6, 3, 6, -89, 6, 14, 6, 88, 6, -78, 6, -1, 6, 2, 6, 7]

Input: [1, 2, 3, 4, 5, 6]

Separator: 9

Output: [1, 9, 2, 9, 3, 9, 4, 9, 5, 9, 6]

[25]:
```
def test(nums, sep):
    L=[]
    for i in range(len(nums)):
        if i==len(nums)-1:
            L.append(nums[i])
        else:
            L.append(nums[i])
            L.append(sep)
    return L
nums = [12, -7, 3, -89, 14, 88, -78, -1, 2, 7]
separator = 6
print("List of numbers:",nums)
print("Separator:",separator)
print("Inject the separator in between each pair of adjacent numbers of the␣
 ↪said list:")
print(test(nums,separator))
```

```
List of numbers: [12, -7, 3, -89, 14, 88, -78, -1, 2, 7]
Separator: 6
Inject the separator in between each pair of adjacent numbers of the said list:
[12, 6, -7, 6, 3, 6, -89, 6, 14, 6, 88, 6, -78, 6, -1, 6, 2, 6, 7]
```

38.8 Write a Python program to start with a list of integers, keep every other element in place and otherwise sort the list.

Input: [2, 5, 6, 3, 1, 4, 34]

Output: [1, 5, 2, 3, 6, 4, 34]

Input: [8, 0, 7, 2, 9, 4, 1, 2, 8, 3]

Output: [1, 0, 7, 2, 8, 4, 8, 2, 9, 3]

Vishal Acharya

[33]:
```
nums=[8, 0, 7, 2, 9, 4, 1, 2, 8, 3]
L1=nums[::2]
L2=nums[1::2]
L1.sort()
L=[]
for i in range(len(L1)):
    if len(L1)==len(L2):
        L.append(L1[i])
        L.append(L2[i])
    else:
        L.append(L1[i])
print(L)
```

```
[1, 0, 7, 2, 8, 4, 8, 2, 9, 3]
```

38.9 Write a Python program to get the single digits in numbers sorted backwards and converted to English words.

Input: [1, 3, 4, 5, 11]

Output: [‘five’, ‘four’, ‘three’, ‘one’]

Input: [27, 3, 8, 5, 1, 31]

Output: [‘eight’, ‘five’, ‘three’, ‘one’]

[40]:
```
def test(nums):
    digits = {None: 'zero', 1: 'one', 2: 'two', 3: 'three', 4: 'four', 5:␣
 ↪'five', 6: 'six', 7: 'seven', 8: 'eight', 9: 'nine'}
    nums.sort()
    L=[digits[i] for i in nums if i>=0 and i<10]
    L.reverse()
    print(L)
nums= [27, 3, 8, 5, 1, 31]
test(nums)
```

```
['eight', 'five', 'three', 'one']
```

38.10 Write a Python program to find the following strange sort of list of numbers: the first element is the smallest, the second is the largest of the remaining, the third is the smallest of the remaining, the fourth is the smallest of the remaining, etc.

Input: [1, 3, 4, 5, 11]

Output: [1, 11, 3, 5, 4]

Input: [27, 3, 8, 5, 1, 31]

Output: [1, 31, 3, 27, 5, 8]

Input: [1, 2, 7, 3, 4, 5, 6]

Vishal Acharya

Output: [1, 7, 2, 6, 3, 5, 4]

[44]:
```
nums = eval(input("Enter list: "))
temp=nums.copy()
L=[]
i=0
while i<len(temp):
    if i%2==0:
        L.append(min(nums))
        nums.remove(min(nums))
        i+=1
    else:
        L.append(max(nums))
        nums.remove(max(nums))
        i+=1
print(L)
```

```
Enter list: [27, 3, 8, 5, 1, 31]
[1, 31, 3, 27, 5, 8]
```

38.11 Write a Python program to find four positive even integers whose sum is a given integer.

Input: n = 100

Output: [94, 2, 2, 2]

Input: n = 1000

Output: [994, 2, 2, 2]

Input: n = 10000

Output: [9994, 2, 2, 2]

Input: n = 1234567890

Output: [1234567884, 2, 2, 2]

[49]:
```
def test(n):
    for a in range(n, 0, -1):
        if a % 2 != 0:
            continue
        for b in range(n - a, 0, -1):
            if b % 2 != 0:
                continue
            for c in range(n - b - a, 0, -1):
                if c % 2 != 0:
                    continue
                for d in range(n - b - c - a, 0, -1):
                    if d % 2 != 0:
                        continue
```

Vishal Acharya

```
                    if a + b + c + d == n:
                        return [a, b, c, d]

n = int(input("Enter a number"))
print("Four positive even integers whose sum is",n)
print(test(n))
```

```
Enter a number1234567890
Four positive even integers whose sum is 1234567890
[1234567884, 2, 2, 2]
```

38.12 Write a python program to implement linear search.

Linear search is a method of finding elements within a list. It is also called a sequential search. It is the simplest searching algorithm because it searches the desired element in a sequential manner. It compares each and every element with the value that we are searching for. If both are matched, the element is found, and the algorithm returns the key's index position.

[50]:
```
def linearsearch(arr, x):
   for i in range(len(arr)):
      if arr[i] == x:
         return i
   return -1
arr = ['t','u','t','o','r','i','a','l']
x = 'a'
print("element found at index "+str(linearsearch(arr,x)))
```

```
element found at index 6
```

38.13 Find three numbers from an array such that the sum of three numbers equal to zero

[51]:
```
# return a list of lists of length 3
def three_Sum(num):
    if len(num)<3:
        return []
    num.sort()
    result=[]
    for i in range(len(num)-2):
        left=i+1
        right=len(num)-1
        if i!=0 and num[i]==num[i-1]:
            continue
        while left<right:
            if num[left]+num[right]==-num[i]:
                result.append([num[i],num[left],num[right]])
                left=left+1
                right=right-1
```

Vishal Acharya

```
                while num[left]==num[left-1] and left<right:left=left+1
                while num[right]==num[right+1] and left<right: right=right-1
            elif num[left]+num[right]<-num[i]:
                left=left+1
            else:
                right=right-1
    return result

nums1=[-1,0,1,2,-1,-4]
nums2 = [-25,-10,-7,-3,2,4,8,10]
print(three_Sum(nums1))
print(three_Sum(nums2))
```

```
[[-1, -1, 2], [-1, 0, 1]]
[[-10, 2, 8], [-7, -3, 10]]
```

38.14 Write a python program to check a sequence of numbers is a geometric progression or not.

[53]:
```
def is_geometric(li):
    if len(li) <= 1:
        return True
    # Calculate ratio
    ratio = li[1]/(li[0])
    # Check the ratio of the remaining
    for i in range(1, len(li)):
        if li[i]/(li[i-1]) != ratio:
            return False
    return True

print(is_geometric([2, 6, 18, 54]))

print(is_geometric([10, 5, 2.5, 1.25]))

print(is_geometric([5, 8, 9, 11]))
```

```
True
True
False
```

[4]:
```
def mirrorChars(input,k):
    # create dictionary
    original = 'abcdefghijklmnopqrstuvwxyz'
    reverse = 'zyxwvutsrqponmlkjihgfedcba'
    dictChars = {}
    for i in range(len(original)):
        dictChars[original[i]]=reverse[i]
```

Vishal Acharya

Vishal Acharya

```
    # separate out string after length k to change
    # characters in mirror
    prefix = input[0:k-1]
    suffix = input[k-1:]
    mirror = ''
    # change into mirror
    for i in range(0,len(suffix)):
        mirror = mirror + dictChars[suffix[i]]
    # concat prefix and mirrored part
    print (prefix+mirror)

input = 'ljietengx'
k = 3
mirrorChars(input,k)
```

```
ljrvgvmtc
```

[5]:
```
test_dict = {'Gfg' : {"a" : 7, "b" : 9, "c" : 12},
'is' : {"a" : 15, "b" : 19, "c" : 20},
'best' :{"a" : 5, "b" : 10, "c" : 2}}
# printing original dictionary
print("The original dictionary is : " + str(test_dict))
# initializing key
temp = "c"
# using keys() and values() to extract values
res = [sub[temp] for sub in test_dict.values() if temp in sub.keys()]
# printing result
print("The extracted values : " + str(res))
```

```
The original dictionary is : {'Gfg': {'a': 7, 'b': 9, 'c': 12}, 'is': {'a': 15,
'b': 19, 'c': 20}, 'best': {'a': 5, 'b': 10, 'c': 2}}
The extracted values : [12, 20, 2]
```

[7]:
```
def Convert(tup, di):
    for a, b in tup:
        di.setdefault(a, []).append(b)
    return di
tups = [("akash", 10), ("gaurav", 12), ("anand", 14),
("suraj", 20), ("akhil", 25), ("ashish", 30)]
dictionary = {}
print (Convert(tups, dictionary))
```

```
{'akash': [10], 'gaurav': [12], 'anand': [14], 'suraj': [20], 'akhil': [25],
'ashish': [30]}
```

Vishal Acharya

T2_Practice Programs_Part2

November 4, 2025

0.1 Python Program For Converting Array Into Zig-Zag Fashion

0.1.1 Given an array of DISTINCT elements, rearrange the elements of array in zig-zag fashion in O(n) time. The converted array should be in form a < b > c < d > e < f.....

[7]:
```
# Python program to sort an array
# in Zig-Zag form

# Program for zig-zag conversion
# of array
def zigZag(arr, n):

    # Flag true indicates relation "<"
    # is expected, else ">" is expected.
    # The first expected relation is "<"
    flag = True

    for i in range(n - 1):

        # "<" relation expected
        if flag is True:

            # If we have a situation like
            # A > B > C, we get A > B < C
            # by swapping B and C
            if arr[i] > arr[i + 1]:
                temp = arr[i]
                arr[i]  = arr[i + 1]
                arr[i + 1] = temp

            # ">" relation expected
        else:

            # If we have a situation like
            # A < B < C, we get A < C > B
            # by swapping B and C
            if arr[i] < arr[i + 1]:
```

```
                arr[i], arr[i + 1] = arr[i + 1], arr[i]
        flag = bool(1 - flag)
    print(arr)
arr = [4, 3, 7, 8, 6, 2, 1]
n = len(arr)
zigZag(arr, n)
```

```
[3, 7, 4, 8, 2, 6, 1]
```

0.2 Python Program for Leaders in an array

Write a program to print all the LEADERS in the array. An element is leader if it is greater than all the elements to its right side. And the rightmost element is always a leader. For example in the array {16, 17, 4, 3, 5, 2}, leaders are 17, 5 and 2. Let the input array be arr[] and length of the array be size.

[10]:
```
# Python Function to print leaders in array

def printLeaders(arr,size):

    for i in range(0, size):
        for j in range(i+1, size):
            if arr[i]<=arr[j]:
                break
        if j == size-1: # If loop didn't break
            print (arr[i])
arr=[16, 17, 4, 3, 5, 2]
printLeaders(arr, len(arr))
```

```
17
5
2
```

0.3 Python Dictionary to find mirror characters in a string

Given a string and a number N, we need to mirror the characters from the N-th position up to the length of the string in alphabetical order. In mirror operation, we change ‘a’ to ‘z’, ‘b’ to ‘y’, and so on.

[2]:
```
def mirrorChars(input,k):
    # create dictionary
    original = 'abcdefghijklmnopqrstuvwxyz'
    reverse = 'zyxwvutsrqponmlkjihgfedcba'
    dictChars = {}
    for i in range(len(original)):
        dictChars[original[i]]=reverse[i]

    # separate out string after length k to change
    # characters in mirror
```

Vishal Acharya

```
    prefix = input[0:k-1]
    suffix = input[k-1:]
    mirror = ''
    # change into mirror
    for i in range(0,len(suffix)):
        mirror = mirror + dictChars[suffix[i]]
    # concat prefix and mirrored part
    print (prefix+mirror)

input = 'ljietengx'
k = 3
mirrorChars(input,k)
```

```
ljrvgvmtc
```

0.4 Program for Equilibrium index of an array

Equilibrium index of an array is an index such that the sum of elements at lower indexes is equal to the sum of elements at higher indexes. For example, in an array A:

for example: Input: A[] = [-7, 1, 5, 2, -4, 3, 0]

Output: 3

3 is an equilibrium index, because:

A[0] + A[1] + A[2] = A[4] + A[5] + A[6]

Input: A[] = [1, 2, 3]

Output: -1

[11]:
```
# Python program to find the equilibrium
# index of an array

# function to find the equilibrium index
def equilibrium(arr):

    # finding the sum of whole array
    total_sum = sum(arr)
    leftsum = 0
    for i, num in enumerate(arr):

        # total_sum is now right sum
        # for index i
        total_sum -= num

        if leftsum == total_sum:
            return i
        leftsum += num
```

Vishal Acharya

```
        # If no equilibrium index found,
        # then return -1
    return -1

# Driver code
arr = [-7, 1, 5, 2, -4, 3, 0]
print ('First equilibrium index is ',
       equilibrium(arr))
```

```
First equilibrium index is  3
```

0.5 Python Program To Find Longest Common Prefix Using Sorting

Problem Statement: Given a set of strings, find the longest common prefix. Example:

Input: [“apple”, “ape”, “april”]

Output: “ap”

[12]:
```
def longestCommonPrefix( a):
    size=len(a)
    if size==0:
        return -1
    a.sort()
    end = min(len(a[0]), len(a[size - 1]))
    i = 0
    while (i < end and a[0][i] == a[size - 1][i]):
        i += 1
    pre = a[0][0: i]
    if pre:
        return pre
    else:
        return -1

print("The longest Common Prefix is :" ,longestCommonPrefix(["lessonplan",␣
 ↪"lesson","lees", "length"]))
```

```
The longest Common Prefix is : le
```

0.6 Python Program for Tower of Hanoi

Tower of Hanoi is a mathematical puzzle where we have three rods and n disks. The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules: 1) Only one disk can be moved at a time. 2) Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack. 3) No disk may be placed on top of a smaller disk. Note: Transferring the top n-1 disks from source rod to Auxiliary rod can again be thought of as a fresh problem and can be solved in the same manner.

Vishal Acharya

Vishal Acharya

[4]:
```python
# Recursive Python function to solve the tower of hanoi

def TowerOfHanoi(n , source, destination, auxiliary):
    if n==1:
        print ("Move disk 1 from source",source,"to destination",destination)
        return
    TowerOfHanoi(n-1, source, auxiliary, destination)
    print ("Move disk",n,"from source",source,"to destination",destination)
    TowerOfHanoi(n-1, auxiliary, destination, source)

# Driver code
n = 4
TowerOfHanoi(n,'A','B','C')
# A, C, B are the name of rods
```

```
Move disk 1 from source A to destination C
Move disk 2 from source A to destination B
Move disk 1 from source C to destination B
Move disk 3 from source A to destination C
Move disk 1 from source B to destination A
Move disk 2 from source B to destination C
Move disk 1 from source A to destination C
Move disk 4 from source A to destination B
Move disk 1 from source C to destination B
Move disk 2 from source C to destination A
Move disk 1 from source B to destination A
Move disk 3 from source C to destination B
Move disk 1 from source A to destination C
Move disk 2 from source A to destination B
Move disk 1 from source C to destination B
```

0.7 Minimum time required to rot all oranges

Given a matrix of dimension M * N where each cell in the matrix can have values 0, 1 or 2 which has the following meaning:

0: Empty cell

1: Cells have fresh oranges

2: Cells have rotten oranges

Determine what is the minimum time required so that all the oranges become rotten. A rotten orange at index (i,j) can rot other fresh oranges which are its neighbours (up, down, left and right). If it is impossible to rot every orange then simply return -1.

Examples:

Input: arr[][C] = { {2, 1, 0, 2, 1}, {1, 0, 1, 2, 1}, {1, 0, 0, 2, 1}};

Output: 2

Vishal Acharya

Explanation: At 0th time frame:

{2, 1, 0, 2, 1}

{1, 0, 1, 2, 1}

{1, 0, 0, 2, 1}

At 1st time frame:

{2, 2, 0, 2, 2}

{2, 0, 2, 2, 2}

{1, 0, 0, 2, 2}

At 2nd time frame:

{2, 2, 0, 2, 2}

{2, 0, 2, 2, 2}

{2, 0, 0, 2, 2}

[5]:
```
# Python3 program to rot all
# oranges when u can move
# in all the four direction
# from a rotten orange
R = 3
C = 5

# Check if i, j is under the
# array limits of row and
# column

def issafe(i, j):

    if (i >= 0 and i < R and
            j >= 0 and j < C):
        return True
    return False

def rotOranges(v):

    changed = False
    no = 2
    while (True):
        for i in range(R):
            for j in range(C):

                # Rot all other oranges
                # present at (i+1, j),
                # (i, j-1), (i, j+1),
```

Vishal Acharya

```
                # (i-1, j)
                if (v[i][j] == no):
                    if (issafe(i + 1, j) and
                            v[i + 1][j] == 1):
                        v[i + 1][j] = v[i][j] + 1
                        changed = True

                    if (issafe(i, j + 1) and
                            v[i][j + 1] == 1):
                        v[i][j + 1] = v[i][j] + 1
                        changed = True

                    if (issafe(i - 1, j) and
                            v[i - 1][j] == 1):
                        v[i - 1][j] = v[i][j] + 1
                        changed = True

                    if (issafe(i, j - 1) and
                            v[i][j - 1] == 1):
                        v[i][j - 1] = v[i][j] + 1
                        changed = True

        # if no rotten orange found
        # it means all oranges rottened
        # now
        if (not changed):
            break
        changed = False
        no += 1

    for i in range(R):
        for j in range(C):

            # if any orange is found
            # to be not rotten then
            # ans is not possible
            if (v[i][j] == 1):
                return -1

    # Because initial value
    # for a rotten orange was 2
    return no - 2

v = [[2, 1, 0, 2, 1],
     [1, 0, 1, 2, 1],
     [1, 0, 0, 2, 1]]
```

```
print("Max time incurred: ",rotOranges(v))
```

```
Max time incurred:  2
```

0.8 Check for balanced parentheses in Python

Given an expression string, write a python program to find whether a given string has balanced parentheses or not.

[8]:
```
# balanced parentheses in an expression
def check(my_string):
    brackets = ['()', '{}', '[]']
    while any(x in my_string for x in brackets):
        for br in brackets:
            my_string = my_string.replace(br, '')
    return not my_string

# Driver code
string = "{[]{()}"
print(string, "-", "Balanced"
      if check(string) else "Unbalanced")
```

```
{[]{()} - Unbalanced
```

0.9 Program to generate all possible valid IP addresses from given string

Given a string containing only digits, restore it by returning all possible valid IP address combinations. A valid IP address must be in the form of A.B.C.D, where A, B, C, and D are numbers from 0-255. The numbers cannot be 0 prefixed unless they are 0.

Examples :

Input: 25525511135

Output: [“255.255.11.135”, “255.255.111.35”]

Explanation:

These are the only valid possible IP addresses.

Input: “25505011535”

Output: []

Explanation:

We cannot generate a valid IP address with this string.

[9]:
```
def solve(s, i, j, level, temp, res):

    if (i == (j + 1) and level == 5):
        res.append(temp[1:])
```

Vishal Acharya

```
    # Digits of a number ranging 0-255 can lie only between
    # 0-3
    k = i
    while(k < i + 3 and k <= j):

        ad = s[i: k + 1]

        # Return if string starting with '0' or it is
        # greater than 255.
        if ((s[i] == '0' and len(ad) > 1) or int(ad) > 255):
            return

        # Recursively call for another level.
        solve(s, k + 1, j, level + 1, temp + '.' + ad, res)

        k += 1
# driver code
s = "25525511135"
n = len(s)
ans = []
solve(s, 0, n - 1, 1, "", ans)
for s in ans:
    print(s)
```

```
255.255.11.135
255.255.111.35
```

Vishal Acharya